

Contents

RVSPOC 2026 复现验证材料·A210 卷 1

两题判定速览 1

 S2602 为什么是 #6 1

 P2601 为什么是 #4 2

 S2602 上游行为比对（2026-09-24 自查） 3

第一部分 P2601 KleidiCV 完整验证 4

第二部分 S2602 LiteRT 完整验证 8

第三部分 A210 双板性能与环境差异 13

第四部分 S2602 精度参考与允许区间 16

RVSPOC 2026 复现验证材料·A210 卷

版本 v2.3 发布日期 2026-09-29 编制：组委会验证侧

覆盖赛题：P2601 KleidiCV (b1)、S2602 LiteRT (b2)，共 10 个有效提交

本卷为详细验证记录；终裁结论见《总览与裁定卷》第八章——P2601 冠军 #4、S2602 冠军 #6

两题判定速览

赛题	有无冠军	结论
S2602 LiteRT	有：#6 (260818)	唯一同时通过精度与时延两项口径
P2601 KleidiCV	有：#4 (260818)	经与赞助方共同评审，综合比评选定 #4（终裁见《总览与裁定卷》第八章）

S2602 为什么是 #6

赛题详情页（<https://rvspoc.org/S2602>）定有两道硬门槛：模型级 Top-1 精度（FP32 与 x86 参考差异 $\leq 0.1\%$ 、INT8 $\leq 1\%$ ）、推理延迟 ≤ 110 ms。

六个提交中，四个精度达标（#6、#3、#4、#1），但按已决的「先严后宽」口径，无提交在逐模型口径下六项时延全部达标，故转整体口径——而「多数达标」与「平均 ≤ 110 ms」两种算法结论一致：

提交	精度	时延达标数	六模型平均时延	是否通过
#6	6/6	4/6	97.0 ms	通过
#3	6/6	3/6	133.7 ms	不通过
#4	6/6	1/6	171.3 ms	不通过
#1	6/6	1/6	298.9 ms	不通过
#7	3/6	6/6	55.8 ms	精度出局
#2	4/6	0/6	177.8 ms	不通过

结论不随算法改变，无「换算法换赢家」的申诉空间。#7 时延虽为全场最快，但三个 FP32 模型输出为随机噪声（Top-1 仅 0.10%–0.13%，等同 1000 类随机猜测），精度 3/6，在「三模型缺一不可」处即出局——速度与正确性须合并判读。

关于 #3 的算子级判据（终裁：按字面逐元素执行，见《总览与裁定卷》第八章）：#3 的 MobileNetV2 INT8 在算子级差分中出现最大 3 LSB 的临界超出，该算子级判据计为不通过、如实记录；其模型级精度 6/6 不受影响，时延未达门槛的总体结论不变。

P2601 为什么是 #4

赛题详情页要求「已实现 ARM Neon 优化的算子中，至少 90% 需提供 RVV 加速版本」「所有算子需通过精度验证」「支持 VLEN 128/256/512 自适应」。

提交	完整 API 套件	VLEN 三档	benchmark 测项	结论
#3（瑞福扫）	4496 用例，零失败	三档全过	372	通过
#4（260818）	4496 用例，零失败	三档全过	372	通过
#1（Leaplab）	不适用，见下	自写 8 项冒烟，三档全过	9	通过，但证据强度弱
#2（RVV 智能）	三档均 SIGSEGV	未通过	381	不达标

本题为移植类，规则按「首位验证成功」排名，即按最后 push 时间先后：#3 为 2026-08-24T05:27:28Z，#4 为 2026-08-29T13:23:38Z，#3 早五天。

但《验收备忘录》§ 5.3 同时规定：「若仍需区分（时间接近或评判组认为必要），再按以下综合比评：benchmark 数据完整性、RVV 覆盖率高低、真机实测质量、代码质量」。2026-09-23 新增的 I420 布局专项（见本卷前节）提供了此前不具备的比评依据——#4 两组解码全部通过，#3 分别失败 96 例与 80 例；编码布局一项两家同样失败 80 例。

评委意见（2026-09-22，P2601-I420-20260922 记录随附）：倾向选择 #4，主要理由为——其一，更符合移植任务的核心目标：共同参考测试中 #4 的 I420 解码全部通过，#3 的编码、解码均存在与原版布局不一致的问题；其二，代码组织更清晰，#4 按功能拆分 RVV 实现、便于审查维护，#3 部分实现集中在大型源文件中；其三，#3 在参数校验、验证材料及部分性能测试中的优势不足以抵消已确认的原版兼容性问题。

终裁（2026-09-29）：经与赞助方共同评审，确认选定 #4 为本题冠军；启用综合比评（而非时间优先）的理由即 I420 布局专项所示的原版兼容性差异——#3 两组解码分别失败 96 例与 80 例，#4 全部一致。启用理由在此留痕。

#1 的计分方式已终裁（见《总览与裁定卷》第八章 2.5）：计入评审、以其真实证据强度参与比较。其结果为通过但证据强度弱于同题通过者，未获评冠军。裁定要点：该提交不修改上游 Arm 源码，而将 RVV 实现独立置于 riscv/library/。其代码本身在三档 VLEN 下全部通过、无质量问题，但赛题指定的 KleidiCV 自带 gtest 套件在技术上无法施加于它——其代码接不入上游树，组委会只能运行其自写的 8 项冒烟测试，而 #3、#4 各运行 4496 个用例；其 benchmark 产出 9 条测项，而同侪为 372 至 381 条。争点在于该路线是否符合「移植 KleidiCV」的题意。

I420/YV12 布局专项（2026-09-23 新增，证据由评委独立提供并经组委会复现）

本项不在组委会原定验证范围内。组委会对 P2601 的正确性判据取自赛题详情页所指的 KleidiCV 自带 gtest 套件，#3 与 #4 在该套件下各 4496 用例、三档 VLEN 全部通过、零失败。但该套件验

证的是与参考实现的数值一致性，不验证输出的内存布局是否与上游 26.03 一致。评委于 2026-09-22 就此另行测试（记录编号 P2601-I420-20260922），发现了该套件覆盖不到的问题。

组委会已独立复现，计数逐项吻合。复现所用源码取自组委会自有的 pr-archive 归档（经整树逐文件比对，与评委所附冻结源码零差异；冻结 SHA 亦与本报告记录一致：#3 为 289ac3fb、#4 为 c59a7eb3）；测试驱动沿用评委的 i420_reference_test.cpp，因其参考函数逐字取自 26.03 原版 test/api/test_yuv420p_to_rgb.cpp 与 rgb_to_yuv420_neon.h，是判定「是否与原版布局一致」的必要基准。评委归档的 39 个文件 SHA-256 校验全部通过。

测试组	#3 RVV	#3 标量	#4 RVV	#4 标量
原始参数解码（160 组）	64 / 96	64 / 96	160 / 0	160 / 0
偶数尺寸解码（480 组）	400 / 80	400 / 80	480 / 0	480 / 0
偶数尺寸编码布局（480 组）	400 / 80	400 / 80	400 / 80	400 / 80

单元格为「通过 / 失败」。上表为组委会复现所得，与评委记录的 summary.json 逐项相同，四档测试进程退出码均为 1。

三点须一并呈现，不可只取其一：

其一，该缺陷与向量化无关，属标量实现的地址计算问题。标量档以 KLEIDICV_ENABLE_RVV=OFF、-march=rv64gc 构建（评委已反汇编确认无 RVV 指令、ELF 属性未声明 V 扩展），其结果与 RVV 档完全相同——组委会复现时进一步发现，两档输出日志的 SHA-256 逐字节一致（#3 两档同为 949f9af1...，#4 两档同为 ca0e871d...）。源码分析与之互证：输入宽 2、高 6、stride 6 时，V 三行字节起点，原版 26.03 为 43/48/49，而 #3 的编码与解码均为 43/44/49。

其二，编码布局一项两家同样失败。#3 与 #4 在该组均为 400/80。差异在解码——#4 两组解码全过，#3 分别失败 96 例与 80 例。评委结论中亦明确「双方编码都有缺陷」。

其三，评委已自行标注该测试的局限，组委会认为应当如实转述：编码参考系「从原始源码规则构造的独立参考」，未在 A210 上执行 Arm Neon 原版二进制，亦未覆盖全部颜色与编码精度；失败集中于 height=6/10 且 padding=4 的参数组合，属可重复的布局问题，非接近阈值的舍入差异。

对组委会自身方法的说明：本项暴露了组委会验证方法的一处盲区——自带测试套件全部通过，不等于移植实现与原版行为一致。该盲区同样可能存在于其他移植类赛题的验证中。组委会未在原定方案中设置「与上游行为比对」这一环节，此处如实记录。

原始材料与组委会复现产物存于 evidence/P2601-I420/。

S2602 上游行为比对（2026-09-24 自查）

I420 事件（P2601）暴露的方法盲区是「自带套件全过，不等于与上游原版行为一致」。S2602 存在同类盲区风险，组委会于 09-24 补做自查：提取六个提交各自 fork 点的上游 LiteRT 源码（#1/#2/#3 为 be3611dc，2026-04-17；#4/#6/#7 为 ea79caff，2026-04-04，前者是后者的后代），构建上游标量参考库，与各提交的向量档输出张量比对。每个提交只与它自己 fork 时的上游比对，不跨版本——否则上游自身的变更会被误记为选手改动。

结果：S2602 不存在 I420 类「底座漂移」问题。24 组比对中，19 组通过的差异均在 1e-12 至 1e-7 量级；5 组超阈值者与此前算子级差分所指提交、所涉模型、数值大小完全相同（#7 的 FP32 为 0.958/1.000，#2 的 INT8 为 159/255 LSB，#3 的 INT8 为 3 LSB）。

提交	同提交差分（标量 vs 向量）	上游比对（原版 vs 向量）	定性
#7	FP32 超: 0.958/1.000	FP32 超: 0.958/1.000	向量化本身缺陷
#2	INT8 超: 159/255 LSB	INT8 超: 159/255 LSB	向量化本身缺陷
#3	INT8 超: 3 LSB	INT8 超: 3 LSB	向量化临界超出
#6 / #1 / #4	全过	全过	与原版行为一致

即六个提交的标量底座均未偏离其 fork 的上游原版——三家有缺陷者，问题全部出在向量化实现本身，而非移植底座的漂移。同时，该比对使 #7 与 #2 的缺陷证据链闭合为三条独立方法（模型级 Top-1 对 x86 参考、同提交标量与向量差分、上游原版比对），三者数值互相咬合。

第一部分 P2601 KleidiCV 完整验证

版本 v1.0 发布日期 2026-09-22 本包 17 份文件共用同一版本号

验证日期: 2026-09-18 对应验收备忘录 § 4.B（可复现性）、§ 4.C（正确性）

平台: A210 板 b1（按 D11 分板方案: P2601 用 b1，该题无数值性能门槛，板速不影响判定）
工具链: 板载原生 gcc-14 (Debian 14.2.0-19)、cmake 3.31.6，两板版本已核对一致
口径: RVV 档 -march=rv64gcv -mabi=lp64d; 标量对照档 -march=rv64gc + KLEIDICV_ENABLE_RVV=OFF
归因原则: 凡失败先判定是组委会构建配置问题还是选手侧问题，两者以对照实验分开

一、结论总览

PR	队伍	完整 API 套件（4496 用例）	VLEN 128/256/512	§ 4.C 正确性
#1	Leaplab	不适用——另起并行库，接不入上游测试套件	自写 8 smoke 三档全过	通过（证据强度见 § 3.1 注）
#2	RVV 智能	三档均 SIGSEGV，崩溃点相同	✗	不达标（另有标量档链接失败）
#3	瑞福扫	4496 × 三档，零失败	✓	通过
#4	260818	4496 × 三档，零失败	✓	通过

四个提交中三个通过，一个（#2）存在两处独立的实质问题。

证据强度不对等的说明: #3 与 #4 均在完整 KleidiCV API 套件（4496 用例）下跑满三档 VLEN，二者对等可比。**#1 因不并入上游树，无法运行该套件**，只能跑其自写的 8 个 smoke 测试（每个输出两行）。备忘录 § 4.C 的正确性判据是「KleidiCV 自带测试套件通过」——该判据对 #1 在技术上无法施加。**这不是 #1 的代码缺陷，但「已验证」的含义确实不同**，与 § 3.1 末尾的分母口径问题同源，一并提请评判组。

二、组委会侧构建配置问题（已修正，记录备查）

在给任何提交定性之前，先排除了三处组委会自身的配置问题。这些问题一度让 #2、#3 的构建同时失败，若照单全收会造成误判：

问题	表现	判定依据	修正
未关闭 examples/C	#2、#3 的 RVV 档同时挂在 kleidicv-c-example，缺 operator new、__cxa_guard_* 等 libstdc++ 符号	两个互不相干的提交同一失败点；且选手文档的构建流程本就带 -DKLEIDICV_BUILD_EXAMPLES=OFF	关闭 examples 后 #3 两档均构建通过
对 #1 误用主 CMake 树	#1 主树无条件构建 ARM kleidicv_neon（硬编码 -march=armv8-a），在 RISC-V 上必然失败	#1 的 README 明言不修改上游树，RVV 实现独立于 riscv/library/	改走其自带的 riscv/library + toolchain + Ninja
缺 riscv64-linux-gnu-g++	#1 的 toolchain 文件按该名查找编译器	板上只有 -gcc 无 -g++	建软链接指向 g++-14

examples/C 是纯 C 程序，用 gcc 驱动链接 C++ 静态库，不会拉入 libstdc++。RVV 实现用到了动态分配等 C++ 运行时设施（标量路径不用），故只在 RVV 档暴露。此项不构成对任何选手的不利结论，但可作为 § 4.E「仓库卫生／可移植性」的观察记录。

三、逐提交结果

3.1 #1 (Leaplab) —— 通过

集成方式：不修改上游 Arm 源码，RVV 实现独立置于 riscv/library/，自带 CMakeLists、toolchain (toolchain-rv64gcv.cmake)、smoke 测试与 Docker 流程。其 README 原文：「Upstream Arm sources are not modified; everything new is added here so the upstream tree stays diff-friendly.」

构建：cmake -S riscv/library -G Ninja -DCMAKE_TOOLCHAIN_FILE=riscv/cmake/toolchain-rv64gcv.cmake，其 toolchain 指定 -march=rv64gcv_zba_zbb_zbs（含 bitmanip）。

注：b1 的设备树未通告 zba/zbb/zbs，但组委会已用手写机器码执行 andn 指令验证硬件实际支持（见《A210 卷》第三部分 § 2.1），故该 march 在 b1 上可正常运行。若无此前置验证，本项会被误判为潜在的非法指令风险。

测试：test_filters、test_exp、test_gray_to_rgba、test_yuv、test_add —— 全部 PASS，零失败。

需评判组注意：P2601 是移植类赛题，RVV 覆盖率分母为 KleidiCV 的 130 个 C API 入口（《总览与裁定卷》第五部分 § 1）。#1 走「不改上游、另起并行库」路线，其算子是否计入该分母、以及「移植 KleidiCV」与「另写一个 RISC-V 库」的界线，属规则解释，组委会不单方认定。

3.2 #2（RVV 智能）——两处独立的实质问题

(a) RVV 档：kleidicv-api-test 段错误崩溃

kleidicv-api-te[60925]: unhandled signal 11 code 0x1
at 0x0000002a31a37531 in kleidicv-api-test[2ab1312000+580000] (内核日志)

崩溃用例：RemapF32/0.RandomNoPadding

隔离实验（带 -g 重建后）：

实验	结果
单跑 RemapF32/0.RandomNoPadding 第 1 次	退出码 139 (= 128 + SIGSEGV)
单跑 第 2 次	退出码 139 —— 稳定复现
--gtest_filter=Remap*	退出码 139，崩于同一用例
--gtest_filter=-Remap*（排除 Remap）	退出码 0，4446 用例全通过，零失败

定性：缺陷精确定位于 Remap FP32 路径，其余功能完好。四重佐证：两次稳定复现、内核日志记录、排除该路径后全绿、以及同板同工具链下 #4 的同一测试 4496 用例零失败（排除测试本身与平台因素）。

(b) 标量档：kleidicv-api-test 链接失败

undefined reference to `kleidicv_count_nonzeros_u8'
undefined reference to `kleidicv_error_t kleidicv::neon::min_max_loc<unsigned char>(...).'

机制：关闭 RVV 后，这些 API 在 RISC-V 上既无 Neon 实现（该目标在 RISC-V 不构建）、又无 RVV 实现（已禁用）、也无标量兜底，符号为空。

对应检查项：备忘录 § 4.E「标量回退实现存在，无 RVV 环境可编译运行」——本项不达标。

另注：#2 的 examples/C 是无条件 add_subdirectory，未提供 KLEIDICV_BUILD_EXAMPLES 选项（#3、#4 均有），故组委会无法通过该选项绕开示例，只能改为指定目标构建。此点归入 § 4.E 构建配置质量，酌情记录。

3.3 #3（瑞福扫）——通过

集成方式：并入 KleidiCV 主 CMake，且含 5 处 CMAKE_SYSTEM_PROCESSOR MATCHES "riscv" 架构门控，在 RISC-V 上自动屏蔽 Neon/SVE2 目标（实测 kleidicv_neon 目标数 = 0）。另提供 scripts/rvspoc_a210_validate.sh 一键验证脚本与 doc/rvspoc/reproduce-a210.md 复现文档。

结果：RVV 档（68 s）与标量档（36 s）均构建通过；各 8 个 smoke 测试（resize / morphology / analysis / yuv / add / lk / lk-pyramid / transform）全部 PASS，零失败。

3.4 #4（260818）——通过，覆盖最完整

结果：

档位	构建	kleidicv-framework-test	kleidicv-api-test
RVV (rv64gcv)	548 s	36 用例 PASS	4496 用例 PASS，零失败
标量 (rv64gc)	594 s	36 用例 PASS	4496 用例 PASS，零失败

两档均通过完整 API 测试套件，是本题覆盖最完整的一份结果。其标量档通过，也反证了 § 3.2(b) 中 #2 的标量链接失败属该提交自身问题，而非平台或组委会配置所致。

三-bis、VLEN 多档验证（§ 4.C 要求，已完成）

两块 A210 实测 VLEN 均为 128（vlenb CSR 读取确认），故 256/512 两档按备忘录用 QEMU -cpu rv64,v=true,vlen=N 补。在 b1（riscv64 原生机）上跑 user-mode QEMU 10.0.13，sysroot 天然正确；先用 vlenb 探针确认 QEMU 确实按请求设置 VLEN（128→16 字节、256→32、512→64），排除空跑。同一份 -march=rv64gcv 二进制在三档下跑同一套测试。

PR	VLEN=128	VLEN=256	VLEN=512
#1	8 项全过	8 项全过	8 项全过
#3	smoke 10 项 + api-test 4496 用例 全过	同左	同左
#4	framework 36 + api-test 4496 用例 全过	同左	同左
#2	api-test SIGSEGV	SIGSEGV	SIGSEGV

#1、#3、#4 的 RVV 代码均不依赖特定向量长度——此类缺陷真机无法暴露（硬件仅 128 档）。
#2 的三档崩溃点完全相同（均停在 RemapF32/0.RandomNoPadding），说明缺陷与向量长度无关，属固定逻辑错误（指针越界／未初始化内存一类），而非 VLEN 边界处理问题。此信息对选手定位有直接价值，建议写入公开通报。

#3 的完整 API 测试补测（公平性纠正，2026-09-18）

初次验证时 #4 跑了 4496 用例、#3 只跑了 8 个 smoke。追查发现差异源于两人的 CMake 默认值：

```
# #3: RISC-V 分支把完整套件额外挡在一个默认关闭的开关之后
if(CMAKE_SYSTEM_PROCESSOR MATCHES "riscv")
  add_subdirectory(test/rvv)           # smoke，默认构建
  if(KLEIDICV_BUILD_FULL_API_TESTS)   # ← 默认 OFF
    add_subdirectory(test)            # 4496 用例
# #4: 默认即构建完整套件
if(KLEIDICV_BUILD_TESTS)              # 默认 ON
  add_subdirectory(test)
```

若照单收下，#3 会被无端拉低。故显式开启 -DKLEIDICV_BUILD_FULL_API_TESTS=ON 补测，跑法与 #4 完全一致（同样排除 CornerCasesLargeLoad、BigSourceImage 两类超大内存用例—— #3 文档亦指出板卡内存扛不住这些用例）。结果：三档各 4496 用例零失败，与 #4 对等。

方法论提示：本届评审中反复出现同一模式——各提交的构建配置差异会让组委会无意中施加不等强度的验证。评审记录应写明「每个提交实际跑了什么测试、为什么」，而不止于「通过/不通过」。

四、未完成项

- 1. benchmark 数据完整性（§ 4.D）：P2601 无数值门槛，考察 kleidicv-benchmark 数据完整性与真机实测质量，尚未采集。

2. **RVV 覆盖率核对** (§ 4.E 门槛项): 分母表已备 (130 个 API), 逐提交核对未做; #1 的计入口径待评判组明确 (见 § 3.1)。

五、留档

内容	路径 (b1 /work/)
构建与测试结果	results/p2601_build.tsv、results/p2601_noexamples.tsv、results/p2601_pr2_targets.tsv
#2 段错误复现	results/pr2_segfv_repro.txt、logs/pr2-segfv.*.log
全部构建与测试日志	logs/ (每提交每档独立日志)
脚本	p2601_v2.sh (逐 PR 构建路径)、p2601_v4.sh (关闭 examples 复验)、p2601_v5.sh (#2 专项)、pr2_repro.sh (段错误复现)
冻结 SHA 核对	/work/rvspoc-P2601-kleidicv-pr*.FROZEN_SHA (与 pr-archive/MANIFEST.tsv 一致)

第二部分 S2602 LiteRT 完整验证

版本 v1.0 发布日期 2026-09-22 本包 17 份文件共用同一版本号

验证日期: 2026-09-18 (进行中) 对应验收备忘录 § 4.C (精度)、§ 4.D (性能)

平台: A210 板 b2 (按 D11: S2602 用 b2——其 cpufreq 可读可锁、performance 调速器就位, 三轮读数分毫不差, 可审计性最好; 代价是 b2 比 b1 慢 12%–30%, 门槛实际更严) 工具链: 板载原生 gcc-14 (Debian 14.2.0-19)、cmake 3.31.6 构建: -march=rv64gcv -mabi=lp64d, TFLITE_ENABLE_XNNPACK=OFF、TFLITE_ENABLE_GPU=OFF 进度: #6、#2 已完成; #1、#7、#3、#4 待跑

一、评测方法 (与 x86 参考的口径对齐)

1.1 预处理只在 x86 做一次——消除 resize 实现差异

FP32 的判定阈值是 **Top-1 差异 ≤ 0.1%** (10,000 张里仅容 10 张之差)。若在板端用 C++ 重写预处理, PIL 的 BILINEAR resize 与任何 C++ 实现的细微差别都足以吃掉这点余量。

故: 10,000 张图在 x86 上用与 eval_top1.py 完全相同的代码 (PIL BILINEAR resize 短边至 256 + center crop 224) 预处理为 224×224×3 uint8 张量, 一次性落盘 (1.5 GB), 传至板端。板端只做归一化 (x-127)/128 与量化 round(norm/scale)+zero——**纯算术, 逐位可复现**。

这样两侧唯一的差异就是**推理引擎本身**, 正是 S2602 要测的量。

- 张量块 blob_sha256: f4f03ab2376a8da0e6c9bf8920afc45861ab46ba525d7330e8e150f75d4bc1d3 (板端已核对一致)
- list_sha256: c7139aaf... 与 本卷第四部分 记录一致

验证效果: #6 与 #2 的三个 FP32 模型 **Top-1 全部与 x86 参考一位不差** (差值 0.00%), 证明该方案确实消除了预处理噪声。

1.2 组委会统一 harness——六个提交横向可比

上游 benchmark_model 在本 LiteRT 快照下编译不通过（见 § 2），故精度与时延均由组委会自写的单一 harness 完成，链接各选手各自构建的 libtensorflow-lite.a。备忘录 § 4.D 只规定指标、未指定工具，此口径已如实记录。

时延指标覆盖 § 4.D 要求：avg / p50 / p95 / std / p95 ÷ avg / throughput / 内存峰值 (VmHWM)。每配置 warmup 10 次后计时 50 次，3 轮。**时延测量独占串行**；精度则 6 模型并行（每进程单线程，实测并行不损耗单进程速度：1357.9 s vs 单独跑 1359.8 s）。

1.3 已知的口径差异（如实记录，不影响判定）

三个 INT8 模型的 Top-1 一致地略高于 x86 参考（+0.04% ~ +0.10%），而三个 FP32 完全精确。原因：x86 参考用 numpy np.round（银行家舍入），板端 harness 用 C++ std::round（逢五远离零），恰好落在 .5 的像素取整方向不同。FP32 路径不经量化取整，故无差异——这一模式本身印证了该解释。

性质：harness 两侧的舍入口径差异，**非选手问题**，且对所有提交一视同仁。差值 0.1% 对 INT8 的 1% 阈值有十倍余量，故如实记录而不重跑。

二、上游 LiteRT 的两处构建缺陷（六个提交一视同仁）

本快照的 CMake 与其自身 fetch 的 TensorFlow v2.21.0-rc0 版本错配，两处均由组委会适配，性质同 S2603 的 -fno-lto：只动构建配置、不碰选手代码。

#	现象	根因	处置
1	tf-lite/kernels/random_ops.cc 找不到 tsl/lib/random/philox_random.h	CMake 写死 TSL_SOURCE_DIR = third_party/xla/third_party/tsl, 但该 TF 版本已将 TSL 迁至 third_party/xla/xla/tsl（原路径下只剩 platform、profiler）	补 -I<build>/tensorflow-src/third_party/xla/xla
2	benchmark_model 编译失败：StatWithPercentiles redeclared with 1 template parameter	该类在 LiteRT 自带 tools/benchmark/stat_with_percentiles.h 与 fetch 来的 TF core/util/stats_calculator.h 中各定义一次，模板参数个数不同	库 libtensorflow-lite.a 不受影响（构建至 97% 才失败）；改用组委会 harness 计时，绕开该工具

三、S2602 #6 (260818)

3.1 精度——6 组全部通过

模型	精度	实测	x86 参考	差值	阈值	判定
MobileNetV1	FP32	58.33%	58.33%	0.00%	≤0.1%	✓
MobileNetV1	INT8	57.22%	57.16%	+0.06%	≤1%	✓
MobileNetV2	FP32	58.84%	58.84%	0.00%	≤0.1%	✓
MobileNetV2	INT8	58.35%	58.25%	+0.10%	≤1%	✓
EfficientNet-Lite0	FP32	61.67%	61.67%	0.00%	≤0.1%	✓
EfficientNet-Lite0	INT8	61.57%	61.49%	+0.08%	≤1%	✓

交叉对照：MobileNetV2 FP32 在单线程、4 线程、并行三种配置下 Top-1 均为 0.588400，证实 Top-1 与线程数无关。

3.2 时延——6 个模型中 4 个达标（取每模型较优线程数）

模型	1 线程	4 线程	较优	110 ms
EfficientNet-Lite0 INT8	144.7	44.7	44.7	✓ 余量 59%
MobileNetV2 INT8	157.4	62.3	62.3	✓ 余量 43%
MobileNetV2 FP32	89.7	219.2	89.7	✓ 余量 18%
MobileNetV1 INT8	220.6	91.0	91.0	✓ 余量 17%
EfficientNet-Lite0 FP32	116.0	245.0	116.0	✗ 仅超 5.5%（边界带，建议复测）
MobileNetV1 FP32	178.2	261.5	178.2	✗ 超 62%

特征：三个 INT8 全部达标且余量充裕，三个 FP32 中两个不达标——RVV 优化主要作用于量化路径。

线程数的影响是决定性的且方向相反：FP32 用 4 线程反而慢 2.4 倍，INT8 用 4 线程快 2.5 倍。门槛口径（线程数、逐模型还是整体）见待决策 D12a / D12b。

数据稳定性：除 MobileNetV1 FP32 单线程三轮极差达 16% 外，其余配置极差均 < 2 ms，p95/avg ≈ 1.002。该模型体积最大（17 MB），若落在判定边界应复测。

四、S2602 #2（RVV 智能）

4.1 精度——2 组归零【待干净重建复验后定性】

模型	精度	实测	参考	判定
MobileNetV1	FP32	58.33%	58.33%	✓ 一位不差
MobileNetV2	FP32	58.84%	58.84%	✓ 一位不差
EfficientNet-Lite0	FP32	61.67%	61.67%	✓ 一位不差
EfficientNet-Lite0	INT8	61.53%	61.49%	✓ +0.04%
MobileNetV1	INT8	1.55%	57.16%	✗ 接近随机
MobileNetV2	INT8	0.10%	58.25%	✗ 等于随机 (1/1000)

失败签名很窄：不是「INT8 全坏」（EfficientNet-Lite0 的 INT8 正常），也不是「MobileNet 全坏」（两者 FP32 均完美），而是 **MobileNet × INT8 这个交叉点**。两模型均大量使用 depthwise separable 卷积，量化路径的 depthwise 内核是首要嫌疑。

支持「选手侧问题」的证据：- 同一 harness、同一模型文件、同一批张量，#6 的库跑出 58.35%，#2 的库跑出 0.10% - #2 自身另外 4 组完全正常 → harness 与数据均无问题 - 小样本复跑（200 张）复现：MobileNetV2 INT8 0/200、MobileNetV1 INT8 6/200

但尚未定性：#2 的库是组委会在多次中断构建的过程中增量攒出的（跨数轮，时间戳 17:22），不能排除产物不一致。已排全新构建树重编复验（pr2_clean_verify.sh，跑 500 张并与 #6 的库对照同一批），结论以该复验为准。

4.2 时延——6 个模型全部不达标

模型	较优配置	中位	vs 110 ms
EfficientNet-Lite0 INT8	4 线程	111.7 ms	超 1.5%
MobileNetV1 INT8	4 线程	130.6 ms	超 19%
MobileNetV2 INT8	4 线程	137.1 ms	超 25%
MobileNetV2 FP32	4 线程	211.4 ms	超 92%
MobileNetV1 FP32	4 线程	246.1 ms	超 124%
EfficientNet-Lite0 FP32	4 线程	230.0 ms	超 109%（原记 393.2 ms/1 线程，系 4 线程数据到齐前的过时值，已于 09-20 交叉核对时更正）

与 #6 的直接对比（MobileNetV2 FP32、单线程）：**#6 为 89.3–91.1 ms，#2 为 308.1–309.3 ms，慢 3.4 倍**。该倍数与精度阶段的耗时差（6535 s vs 1945 s）完全一致，说明是稳定的性能特征。

注：#2 是 4 线程全面更优，而 #6 是 FP32 用单线程更优——各提交的最优线程配置本就不同，这是 D12a 不能一刀切的实据。

四-bis、S2602 #7 (liuyd-dev) ——FP32 路径失效

组委会侧的编译器误用（已修正，必须记录）

初次构建 #7 失败：

```
error: too few arguments to function
'__riscv_vsmul_vx_i32m4(vint32m4_t, int, unsigned int, long unsigned int)'
（vssra / vnclipu 同）
```

这是组委会的错误。#7 的 PR 描述明确写有：

环境要求：riscv64-linux-gnu-g++ 13.3.0 即可（不需要 GCC 14）、本项目的定点舍入通过 3 参数 **vnclip** 实现，不依赖 GCC 14 才引入的……

vsmul、vssra、vnclipu 这组定点 intrinsics 自 GCC 14 起增加了 vxrm 舍入模式参数。选手预见到该变更并写入文档，而组委会统一使用 gcc-14 构建，故编译失败。**改用 gcc-13.3.0 后构建通过（6136 s）。**

此后流水线已改为**按各提交文档声明的编译器逐个构建**。系统核查六个提交的声明：#1 GCC 14、#2 **GCC 15.2.0**（Debian trixie 不可得，见 § 4.3）、#7 GCC 13.3.0、#3/#4/#6 未指定（沿用平台默认 gcc-14）。

精度——3/6，三个 FP32 全部归零

模型	精度	实测	参考	判定
MobileNetV1	INT8	57.22%	57.16%	✓
MobileNetV2	INT8	58.35%	58.25%	✓
EfficientNet-Lite0	INT8	61.57%	61.49%	✓
MobileNetV1	FP32	0.13%	58.33%	✗ 随机
MobileNetV2	FP32	0.10%	58.84%	✗ 随机
EfficientNet-Lite0	FP32	0.11%	61.67%	✗ 随机

独立三重对照复核（各 200 张，同一 harness、同一批张量）：

实验	结果
#7 的库 × MobileNetV2 FP32	0/200 (0.0%)
#7 的库 × MobileNetV2 INT8	145/200 (72.5%) ✓
#6 的库 × 同一 FP32 模型	147/200 (73.5%) ✓

harness 与数据均被排除；且本次构建使用选手自己声明的 **gcc-13.3.0**，编译器版本因素亦被排除。

时延——6/6 达标，但 FP32 三项不应采信

模型	较优配置	中位	余量	可采信
MobileNetV2 INT8	4 线程	20.6 ms	81%	✓
EfficientNet-Lite0 INT8	4 线程	22.1 ms	80%	✓
MobileNetV1 INT8	4 线程	25.5 ms	77%	✓
MobileNetV2 FP32	1 线程	67.3 ms	39%	✗ 输出错误
EfficientNet-Lite0 FP32	1 线程	92.6 ms	16%	✗ 输出错误
MobileNetV1 FP32	1 线程	106.7 ms	3%	✗ 输出错误

三个 FP32 的时延均为全场最快（分别比 #6 快 25%、20%、40%），但其输出为随机噪声。**速度与正确性必须合并判读**：在 FP32 路径不产生正确结果的前提下，这三项时延数字不构成有效的性能证据。三个 INT8 的时延则对应正确输出，可以采信，且余量充裕（≥77%）。

评审记录要点：本题时延与精度分属两个阶段，组委会为尽快取得门槛判定曾将时延提前。该安排本身合理，但在精度结果到达前不得就「是否达标」下结论——#7 即为实例：仅看时延会得出「首个六模型全线达标」的错误结论。

五、留档

内容	路径（b2 /work/s2602/）
精度结果	results/s2602_accuracy.tsv、results/s2602_accuracy_1thread_crosscheck.tsv

时延结果	results/s2602_latency.tsv
构建结果	results/s2602_build.tsv
预处理张量与清单	tensor_dump/{tensors_u8.bin,labels.i32,manifest.json}
harness 源码	accuracy_harness.cc
流水线脚本	run_v7.sh; #2 复验 pr2_clean_verify.sh
全部日志	logs/
冻结 SHA 核对	src/rvspoc-S2602-litert-pr*.FROZEN_SHA

第三部分 A210 双板性能与环境差异

版本 v1.0 发布日期 2026-09-22 本包 17 份文件共用同一版本号

测量日期：2026-09-17 对应验收备忘录 § 2「验证环境定版」、§ 4.D「性能」

结论先行：**两块板性能差异显著 (12%–30%)，不可跨板比对**。主因是主频差：b1 $\approx$ 2.29 GHz，b2 锁定 1.896 GHz。S2602 的 110 ms 是**绝对值硬门槛**，该差异足以直接决定过与不过。

一、性能实测

无编译器可用（见 § 3），故基准全部基于两板**完全相同**的 CPython（3.13.5 (main, Jun 25 2025) [GCC 14.2.0]，逐字节同版本）。每项 5 轮取中位数。

基准	b1 中位	b2 中位	b2 相对 b1
整数运算循环	2.1219 s	2.3692 s	慢 11.7%
浮点运算循环	0.4191 s	0.5058 s	慢 20.7%
字节扫描（贴近协议解析／图像行遍历）	0.1390 s	0.1808 s	慢 30.1%
内存拷贝	0.0291 s	0.0309 s	— 不采用

内存拷贝项两板 CV 均达 42%–45%（远超噪声容忍度），数据不可靠，不作为结论依据。其余三项 CV 均 $\leq$ 5.4%，可用。

有效主频（perf 硬件计数器，各 3 轮）

板	实测有效频率	cpufreq 接口	调速器
b1	2.284 / 2.298 / 2.298 GHz	不存在	无法读取，无法锁定
b2	1.896 / 1.896 / 1.896 GHz（分毫不差）	完整	performance，可选 500M–1896M

频率比 $2.293 / 1.896 = 1.21$ ，与整数、浮点两项的时间差基本吻合；字节扫描项差异更大 (30%)，说明除主频外还有微架构或内存子系统的差异。

二、两板并非同构——四处差异

2.1 ISA 通告不同，但硬件实为一致

板	/proc/cpuinfo ISA
b1	rv64imafdcv_sscfpmf_svpbmt
b2	rv64imafdcv_zicntr_zicsr_zifencei_zihpm_**zba_zbb_zbs**_sscfcvpmf_svpbmt

b1 未通告 bitmanip (zba/zbb/zbs)。为确认是硬件差异还是设备树通告差异，板上无编译器，遂以手写机器码直接执行一条 Zbb 指令探测：

```
andn a0, a0, a1    → 0x40B57533
ret                → 0x00008067
输入 (0xFF, 0x0F) 期望 0xF0
```

板	结果
b1	0xF0 PASS, exit=0
b2	0xF0 PASS, exit=0

结论：两块板硬件都支持 Zbb，差异仅在设备树通告。

但这个差异仍会影响评审：凡做运行时探测（hwprobe / cpuinfo 解析）的提交，在 b1 上会回落到标量路径、在 b2 上走 bitmanip 路径——同一份代码在两板上跑的是不同路径，性能自然不可比，正确性覆盖面也不同。编译期选择的提交则不受影响。

2.2 VLEN 两板相同，但只有 128

板	VLENB	VLEN
b1 / b2	16	128

对 P2601 的直接影响：备忘录 § 4.C 要求「VLEN 128/256/512 各验证一遍」。硬件只有 128 档，256 与 512 两档必须全部走 QEMU。两块板均未安装 QEMU；本机有 qemu-riscv64 10.0.11，两档需在本机完成。

2.3 软件状态不一致

项	b1	b2
内核	6.6.0 (perf 构建 6.6.g542b0561c276)	6.6.0 (perf 构建 6.6.g66ddf041de41)
OpenSSL	3.5.6	3.5.1
发行版 / Python	Debian 13 trixie / CPython 3.13.5	同左（一致）
内存	7,925,072 kB	7,920,860 kB

内核主版本相同但构建号不同，OpenSSL 小版本不同。两板尚未做过环境定版快照（备忘录 § 2 遗留项）。

2.4 两块板主机名都是 zhihe

无法靠 hostname 区分身份，脚本与日志必须以 ssh 别名 b1 / b2 标注来源，否则留档无法追溯。

三、开工前必须解决的两个障碍

- 1. 两块板都没有编译器——gcc、cc、g++、clang、make、cmake 全部缺失。选手代码无法在板上原生构建。sudo 免密可用，可 apt 安装；或改为交叉编译。**装什么版本会直接影响结论**（LX5000 阶段已有多个提交必须 gcc-14 才能编过），建议装 gcc-14 并在环境快照中定版。
- 2. 根文件系统仅 19 GB（overlay，已用 177 MB）。LiteRT 与 KleidiCV 的完整构建树加模型与 ImageNetV2（10,000 张）可能吃紧，需先确认余量或挂载外部存储。

四、关于 load average 的说明（避免误判）

两板 load average 长期稳定在 10.01，但 CPU 实际空闲。原因是平台有 10 个 bmu_* 内核线程常驻 D（不可中断睡眠）状态：

```
bmu_ddr0 / bmu_ddr1 / bmu_gpu0 / bmu_npu0 / bmu_pcie0 /  
bmu_usb0 / bmu_vo0 / bmu_vi0 / bmu_vp0 / bmu_peri0
```

D 状态计入 load 但不占 CPU。这是平台常态，不代表板子在忙，后续排期与监控不应据此认为板子被占用。

五、排期建议（待确认，另见决策项 D11）

核心原则：同一赛题的所有提交必须在同一块板上测完，绝不跨板比对。

建议按赛题分板，两题并行且互不干扰：

赛题	建议板	理由
S2602 LiteRT	b2	有 110 ms 绝对值硬门槛，测量必须可审计：b2 频率可读、可锁，performance 调速器已就位，三轮读数分毫不差；b1 无 cpufreq，无法自证测量期间频率稳定
P2601 KleidiCV	b1	该题无数值性能门槛（备忘录 § 4.D：「无数值门槛；考察数据完整性与真机实测」），排名依最后 push 时间与综合比评，故板速不影响判定；b1 更快，可省时间

两点风险须提请评判组注意：

- 1. b2 是慢的那块，用它测 S2602 意味着 110 ms 门槛实际更严。若有提交落在 110 ms ±10% 区间内，建议在 b1 复测并由评判组裁定采信口径（→ D11）。
- 2. 凡含运行时 ISA 探测的提交，在 b1 与 b2 上会走不同代码路径（§ 2.1）。此类提交的正确性验证应与计时在同一块板上完成，不可分开。

六、开工过程中修复的环境问题（2026-09-18）

装工具链与启动验证时，两块板暴露了一系列开箱即用性差异，全部已修复并记录，供环境定版快照与后续复核使用。

#	板	问题	处置
1	b2	/etc/resolv.conf 无 nameserver，DNS 全挂，apt 无法安装任何包	写入 nameserver 192.168.31.1（与 b1 一致，即本机默认网关）
2	b1	/home 是独立的 26 MB 分区且已满；17 GB 的 overlay 挂在 /	改用 /work（overlay 上）作为统一工作目录，两板路径一致

3	b2	无 <code>/etc/ssl</code> 目录、零 CA 证书，所有 HTTPS 失败 (TCP/443 本身可通)	从 b1 复制 <code>ca-certificates.crt</code> 。未动 held 的 openssl 包 (b2 上 openssl 被 hold，故版本停在 3.5.1)
4	b1	缺 <code>riscv64-linux-gnu-g++</code> (只有 <code>-gcc</code>)，P2601 #1 的 <code>toolchain</code> 文件按该名查找编译器	建软链接指向 <code>g++-14</code>
5	两板	无任何编译器/构建工具	安装 <code>gcc-14 g++-14 make cmake git pkg-config</code> (b1 另装 <code>ninja-build</code>)； 两板版本已核对一致 ：gcc 14.2.0-19、cmake 3.31.6、git 2.47.3

对环境定版的意义：这些差异说明两块板并非同一镜像。上表的修复使二者在工具链层面对齐，但 b2 的 `openssl` 仍被 hold (3.5.1 vs b1 的 3.5.6)，`/home` 分区结构也不同。快照文档须如实记录，不可宣称两板同构。

七、P2601 四个提交的集成方式差异（构建时发现，与 § 4.E 代码审查相关）

同一套 KleidiCV，四位选手的 RISC-V 集成方式完全不同，直接影响能否在验证平台原生构建：

PR	集成方式	原生构建	说明
#1	不改上游树，RVV 实现另置于 <code>riscv/library/</code> ，自带 CMake、 <code>toolchain</code> 、 <code>smoke</code> 测试与 Docker 流程	主树不可原生构建 (<code>kleidicv_neon</code> 无条件创建且硬编码 <code>-march=armv8-a</code> ，无开关可关)；须走其 <code>riscv/library</code> 独立树	其 README 明言「Upstream Arm sources are not modified」。另其 <code>toolchain</code> 支持 <code>VLEN</code> 环境变量驱动 QEMU 多档验证
#2	并入主 CMake	可	实测 <code>configure</code> 通过、 <code>kleidicv_neon</code> 目标数 = 0
#3	并入主 CMake，且有 5 处 CMAKE_SYSTEM_PROCESSOR MATCHES "riscv" 架构门控	可	另提供 <code>scripts/rvspoc_a210_validate.sh</code> 一键验证脚本与 A210 复现文档
#4	并入主 CMake，另提供交叉编译 <code>toolchain</code> 文件	可	<code>toolchain</code> 文件要求 <code>riscv64-unknown-linux-gnu-*</code> 前缀工具链（板上为 Debian 原生命名，故原生构建时不使用该文件）

需评判组注意：P2601 是移植类赛题，RVV 覆盖率的分子是 KleidiCV 的 130 个 C API 入口（见《总览与裁定卷》第五部分 § 1）。#1 采用「不改上游、另起并行库」的路线，其算子是否计入该分母、以及「移植 KleidiCV」与「另写一个 RISC-V 库」的界线，属规则解释范畴，**组委会不单方认定**。

第四部分 S2602 精度参考与允许区间

生成日期：2026-09-04 对应验收备忘录 § 4C、§ 9

赛题规则：**FP32 模型级 Top-1 与 x86 参考差异 $\leq 0.1\%$ ，INT8 $\leq 1\%$** 。本报告给出组委会的 x86 参考值，是判定选手 RISC-V 实现精度是否达标的唯一基准。

一、测试集（定版）

项 | 值 |
版本 v1.0 发布日期 2026-09-22 本包 17 份文件共用同一版本号
|—|—| 数据集 | **ImageNetV2 matched-frequency** (Recht et al. 2019) || 规模 | 10,000 张 / 1000 类 / 每类 10 张 || 来源 | HuggingFace vaishaal/ImageNetV2 (公开, 无需授权) || 清单文件 | s2602-x86-ref/val10000.txt || list_sha256 | c7139aafe1d3eb76f5be150461dde3bf551bbbd3e127c8b03616191ddfb2c4c0 || images_sha256 | 457d89237761a290013c469a506e6a8a7ea1859be0c2603f19e8df7fc483f338 |

为何不用 ImageNet val: 组委会无授权拷贝。判定依据是「选手结果与 x86 参考之差」, 两侧跑同一批图, 测试集绝对精度不影响判定, 故替代成立; 且 ImageNetV2 公开可下载, 选手能自行复现核验参考值, 减少申诉空间。

为何用 10,000 张而非 1,000 张: 阈值 0.1% 在 1,000 张时等于「仅允许 1 张判别不同」, 边界完全取决于单张图的运气; 10,000 张时 0.1% = 10 张, 判定才有区分度。

二、x86 参考值

运行环境: x86_64, ai-edge-litert (LiteRT 官方 Python 运行时), 4 线程。

模型	精度	Top-1 参考值	正确数	模型 SHA-256 (前 16 位)
MobileNetV1 1.0 224	FP32	58.33%	5,833	d95b3c5ea86750ce
MobileNetV1 1.0 224	INT8	57.16%	5,716	ecc3a67c47c5a609
MobileNetV2 1.0 224	FP32	58.84%	5,884	9f3bc29e38e90842
MobileNetV2 1.0 224	INT8	58.25%	5,825	f08d447cde49b4e0
EfficientNet-Lite0	FP32	61.67%	6,167	b92051d5c20200b3
EfficientNet-Lite0	INT8	61.49%	6,149	31c18ecf5cfb7bdd

完整结果 (含每个模型的输入量化参数): s2602-x86-ref/reference/*.json

合理性交叉验证

各模型在 ImageNetV2 上相对原版 ImageNet val 的掉幅:

模型	原 val 公布值	本参考值	掉幅
MobileNetV1 FP32	70.9%	58.33%	12.6
MobileNetV1 INT8	70.0%	57.16%	12.8
MobileNetV2 FP32	71.8%	58.84%	13.0
MobileNetV2 INT8	70.8%	58.25%	12.5
EfficientNet-Lite0 FP32	75.1%	61.67%	13.4
EfficientNet-Lite0 INT8	74.4%	61.49%	12.9

六个模型掉幅集中在 12.5–13.4 的狭窄区间, 与文献报告的 ImageNetV2 掉幅 (约 11–14 点) 一致, 说明评测链路 (预处理、标签对齐、量化处理) 正确。

三、判定表（评审直接使用）

选手在 A210 上跑同一测试集、同一模型，Top-1 与下表参考值比较：

模型	参考 Top-1	允许差异	允许正确数区间
MobileNetV1 FP32	58.33%	±0.1%	5,823 – 5,843
MobileNetV2 FP32	58.84%	±0.1%	5,874 – 5,894
EfficientNet-Lite0 FP32	61.67%	±0.1%	6,157 – 6,177
MobileNetV1 INT8	57.16%	±1%	5,616 – 5,816
MobileNetV2 INT8	58.25%	±1%	5,725 – 5,925
EfficientNet-Lite0 INT8	61.49%	±1%	6,049 – 6,249

三个模型缺一不可（规则原文：测试模型不少于 MobileNetV1、MobileNetV2、EfficientDet-Lite0）。

四、评测口径（选手须使用同一套）

- **预处理**：短边缩放至 256（双线性）→ 中心裁剪 224 → 归一化 $(x - 127.0) / 128.0$
- **INT8 输入**：必须按模型自带量化参数换算，不可直接喂原始 uint8 像素
 - $q = \text{clip}(\text{round}(\text{normalized} / \text{scale}) + \text{zero_point}, 0, 255)$
 - MobileNet 量化模型 $\text{scale}=1/128$ 、 $\text{zero}=128$ ，两种喂法结果相同；**EfficientNet-Lite0 INT8 是 $\text{scale}=0.012566$ 、 $\text{zero}=131$** ，直接喂原始像素会把输入拉伸约 1.6 倍，Top-1 损失约 4 个点（组委会首次评测即踩此坑，见 reference-v1-raw-uint8/）
 - 评审时若选手 INT8 精度异常偏低，应优先排查此项，而非直接判不达标
- **标签对齐**：输出 1001 类的模型（MobileNet 系列带 background 类）需 $\text{argmax} - 1$
- **评测脚本**：s2602-x86-ref/eval_top1.py（可随公示材料开源）

五、留档

内容	路径
评测脚本	s2602-x86-ref/eval_top1.py、make_list.py、run_all.sh
测试集清单与 hash	s2602-x86-ref/val10000.txt、val10000.txt.meta.json
模型与校验和	s2602-x86-ref/models/、TFLITE_SHA256SUMS.txt
参考结果	s2602-x86-ref/reference/*.json
作废结果（量化喂入错误）	s2602-x86-ref/reference-v1-raw-uint8/（保留备查）

六、备注：规则文本笔误

赛题表写「EfficientDet-Lite0」，但所附下载链接实为 **EfficientNet-Lite0**（图像分类模型）。EfficientDet 是目标检测模型，没有 Top-1 指标，与规则的 Top-1 判定不自洽。以链接为准执行（即 EfficientNet-Lite0），公示材料中需说明此口径。